

Data Analytics · Python Visualization Reference Guide

Matplotlib & Jupyter Notebook — Practice 7.4

Line Chart · Scatter Plot · Pie Chart · Bar Chart · Formatting · Chart Properties

How to Use This Guide

A Complete Reference + Practice Guide

This guide covers every major Matplotlib chart type used in data analytics. For each chart you will find: the anatomy of the chart (what every part is called), a complete working code template you can use immediately, a property reference table covering every formatting option, and a practice task to complete in your own Jupyter Notebook.

Work through each section in order — by the end you will be able to build a complete visual analytics dashboard from any Pandas DataFrame.

Setup

Create a new notebook: DA_Guide_7_5.ipynb

Add this import cell ONCE at the top — run it before any chart cell:

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
%matplotlib inline
```

The sample DataFrame used throughout this guide is defined in Section 1.

The Universal 5-Step Chart Pattern

Every Matplotlib chart — regardless of type — follows the same five steps. Learn this pattern once and apply it to every chart you ever build.

Step	Action	Code	Why it matters
1	Create canvas	<pre>fig, ax = plt.subplots(figsize=(10,6))</pre>	Sets the drawing area size in inches
2	Draw the data	<pre>ax.plot() / ax.bar() / ax.scatter() / ax.pie()</pre>	Chooses the chart type and plots values
3	Label the chart	<pre>ax.set_title() / ax.set_xlabel() / ax.set_ylabel()</pre>	Every chart must have a title and axis labels
4	Add extras	<pre>ax.legend() / ax.grid() / ax.axhline()</pre>	Legend, gridlines, reference lines

5

Render

```
plt.tight_layout();
plt.show()
```

Removes overlap and displays the chart



Section 1 — Imports & Sample DataFrame

Create the dataset used in every chart throughout this guide

Run this Code cell FIRST in your notebook — every subsequent chart cell depends on `df_sales` and `months_cols` being defined.

In []: Imports and sample DataFrame — run this first

```
# -----
# SECTION 1 — Imports and shared DataFrame
# Run this cell before any other cell in the guide
# -----

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
%matplotlib inline # charts appear inside notebook cells

# — Sample DataFrame: 5 regions × 6 months of sales data -----
df_sales = pd.DataFrame({
    'region' : ['North', 'South', 'East', 'West', 'Central'],
    'jan'     : [12400, 9800, 15200, 8700, 11300],
    'feb'     : [13100, 10500, 14800, 9200, 12100],
    'mar'     : [14500, 11200, 16300, 10100, 13400],
    'apr'     : [13800, 10900, 15700, 9600, 12800],
    'may'     : [15200, 11800, 17100, 10400, 14200],
    'jun'     : [16000, 12400, 18500, 11000, 15600]
})

months_cols = ['jan', 'feb', 'mar', 'apr', 'may', 'jun']
df_sales['total_sales'] = df_sales[months_cols].sum(axis=1)

print(df_sales)
```

Out []: output

	region	jan	feb	mar	apr	may	jun	total_sales
0	North	12400	13100	14500	13800	15200	16000	85000
1	South	9800	10500	11200	10900	11800	12400	66600
2	East	15200	14800	16300	15700	17100	18500	97600
3	West	8700	9200	10100	9600	10400	11000	59000
4	Central	11300	12100	13400	12800	14200	15600	79400



Section 2 — Single-Line Chart

Track one region's sales trend over time

2a. Anatomy of a Line Chart

Chart Part	What it is	Set with
Line	The drawn path connecting data points	<code>ax.plot(x, y)</code>
Marker	Symbol drawn at each data point (circle, square...)	<code>marker='o'</code> or <code>marker='s'</code>
Line style	Solid, dashed, dotted	<code>linestyle='--'</code> or <code>ls=':'</code>
Line width	Thickness of the line in points	<code>linewidth=2</code> or <code>lw=2</code>
Line colour	Hex code, name, or shorthand	<code>color='#2196F3'</code> or <code>color='blue'</code>
X-axis	Horizontal axis — usually time or category	<code>ax.set_xlabel('Month')</code>
Y-axis	Vertical axis — usually numeric values	<code>ax.set_ylabel('Sales (£)')</code>
Title	Chart heading	<code>ax.set_title('Monthly Sales')</code>
Reference line	Horizontal or vertical benchmark	<code>ax.axhline(y=value, ls='--')</code>

2b. Complete Code Template — Single Line Chart

In []: *Single line chart — North region Jan to Jun*

— Single Line Chart — one region's monthly trend —————

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
north = df_sales.loc[df_sales['region'] == 'North',
months_cols].values[0].tolist()

# Step 1 — Create canvas
fig, ax = plt.subplots(figsize=(10, 5))

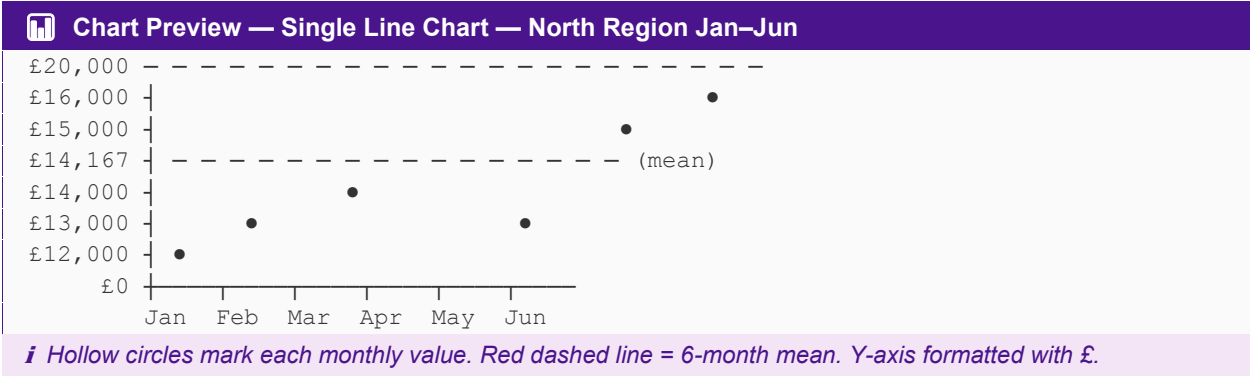
# Step 2 — Draw the line
ax.plot(months, north,
        color='#2196F3',    # line colour
        linewidth=2.5,      # line thickness
        marker='o',         # circle at each data point
        markersize=8,
        markerfacecolor='white', # hollow marker fill
        markeredgewidth=2,
        linestyle='-',       # solid line (use '--' for dashed)
        label='North')
```

```
# Step 3 – Labels and title
ax.set_title('North Region – Monthly Sales Trend H1 2024',
            fontsize=14, fontweight='bold', pad=12)
ax.set_xlabel('Month', fontsize=12)
ax.set_ylabel('Sales (£)', fontsize=12)

# Step 4 – Formatting extras
ax.yaxis.set_major_formatter(
    mtticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax.grid(axis='y', alpha=0.3, linestyle='--')
ax.set_ylim(0, 20000)
# Add a mean reference line
mean_val = sum(north) / len(north)
ax.axhline(mean_val, color='red', ls='--', alpha=0.6, label=f'Mean
£{mean_val:,.0f}')
ax.legend(fontsize=10)

# Step 5 – Render
plt.tight_layout()
plt.show()
```

Out []: output
[Line chart rendered – North region, Jan to Jun]
Values: £12,400 → £13,100 → £14,500 → £13,800 → £15,200 → £16,000
Mean reference line at £14,167 (red dashed)
Y-axis formatted: £0 – £20,000 with £ and comma separators



2c. Single Line Chart — Property Reference

Property / Argument	Example Value(s)	What it controls
color=	'#2196F3' / 'steelblue' / 'b'	Line and marker colour (hex, name, or letter)
linewidth= / lw=	1.0 / 2.5 / 3	Thickness of the drawn line in points
linestyle= / ls=	'-' / '--' / ':' / '-.'	Solid, dashed, dotted, dash-dot
marker=	'o' / 's' / '^' / 'D' / '*' / 'x'	Shape drawn at each data point

<code>markersize= / ms=</code>	5 / 8 / 12	Diameter of the marker symbol
<code>markerfacecolor=</code>	'white' / '#FF0000'	Fill colour inside the marker
<code>markeredgewidth=</code>	1 / 2 / 3	Border thickness of the marker
<code>alpha=</code>	0.3 / 0.7 / 1.0	Transparency (0=invisible, 1=opaque)
<code>label=</code>	'North Region'	Text shown in the legend for this line
<code>ax.axhline(y=,ls=,color=)</code>	<code>axhline(14000, ls='--', color='red')</code>	Horizontal reference line at given y
<code>ax.axvline(x=,ls=,color=)</code>	<code>axvline(2, ls=':', color='gray')</code>	Vertical reference line at given x
<code>ax.set_ylim(min,max)</code>	<code>set_ylim(0, 20000)</code>	Fix y-axis range (prevents auto-scaling)

1

Task

Practice — Single Line Chart

In DA_Guide_7_5.ipynb add a new Code cell and build a single line chart for the EAST region.

Requirements:

- Use a different colour from the example (try '#FF9800' for orange)
- Use a square marker `marker='s'` with a solid fill
- Add a dashed mean reference line in blue
- Format the y-axis with £ and comma separators
- Set the title: 'East Region — Monthly Sales Trend H1 2024'

Expected: upward-trending line from £15,200 to £18,500 — the highest of all regions



Section 3 — Multi-Line Chart

Compare all five regions on one chart

3a. What changes with multiple lines



Key Concept

A multi-line chart uses the same `ax.plot()` call — but inside a for loop, once per data series.

Each call to `ax.plot()` adds ONE line to the axes object `ax`.

Call `ax.legend()` ONCE after the loop — Matplotlib collects all `label=` values automatically.

Use a list of distinct colours and markers to visually separate each line.

3b. Complete Code Template — Multi-Line Chart

In `[ ]`: Multi-line chart — all 5 regions on one chart

```
# — Multi-Line Chart — one line per region —————

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
colours = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']
markers = ['o', 's', '^', 'D', '*']

# Step 1 — Create canvas (wider for multi-line readability)
fig, ax = plt.subplots(figsize=(12, 6))

# Step 2 — Loop: draw ONE line per region
for i, row in df_sales.iterrows():
    ax.plot(months,
            row[months_cols], # the 6 monthly values for this region
            color=colours[i],
            marker=markers[i],
            linewidth=2.2,
            markersize=7,
            label=row['region']
    )

# Add a dashed grey line at the overall monthly mean
overall_mean = df_sales[months_cols].values.mean()
ax.axhline(overall_mean, color='gray', ls='--',
           alpha=0.6, linewidth=1.5,
           label=f'Overall mean £{overall_mean:,.0f}')

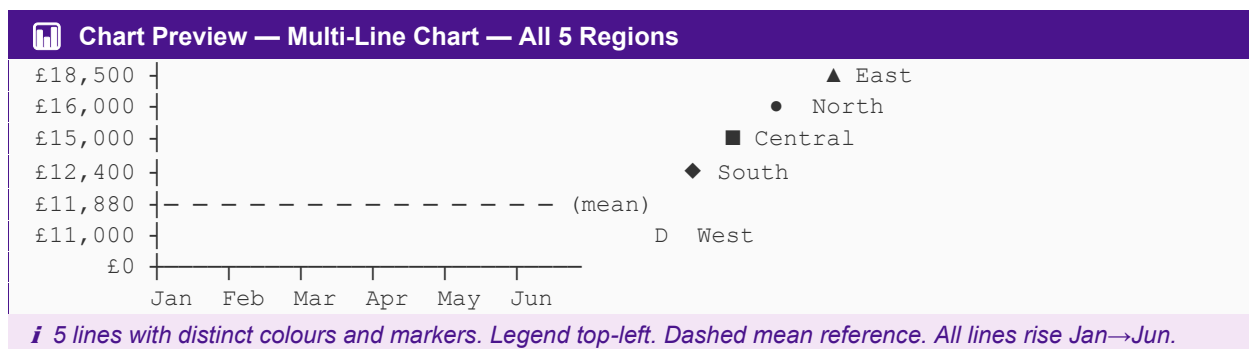
# Step 3 — Labels and title
ax.set_title('Monthly Sales by Region — H1 2024',
            fontsize=15, fontweight='bold', pad=14)
ax.set_xlabel('Month', fontsize=12)
ax.set_ylabel('Sales (£)', fontsize=12)

# Step 4 — Formatting
ax.yaxis.set_major_formatter(
    mtticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax.legend(loc='upper left', fontsize=10, framealpha=0.9)
ax.grid(alpha=0.25, linestyle='--')

# Step 5 — Render
plt.tight_layout()
plt.show()
```

Out[:]: output

```
[5-line chart rendered — all regions on one canvas]
East (orange ▲): highest — rising from £15,200 to £18,500
North (blue ●): second — rising from £12,400 to £16,000
Central (red ■): mid — rising from £11,300 to £15,600
South (green s): fourth — rising from £9,800 to £12,400
West (purple D): lowest — rising from £8,700 to £11,000
Grey dashed mean line at £11,880
All 5 regions trend upward across H1.
```



3c. Multi-Line Chart — Property Reference

Property / Argument	Example Value(s)	What it controls
<code>ax.legend(loc=)</code>	'upper left' / 'best' / 'lower right'	Legend position; 'best' auto-selects least cluttered
<code>ax.legend(framealpha=)</code>	0.5 / 0.9 / 1.0	Legend box transparency (0=see-through, 1=solid)
<code>ax.legend(fontsize=)</code>	8 / 10 / 12	Font size of legend text
<code>ax.legend(title=)</code>	'Region'	Heading text inside the legend box
<code>ax.legend(ncol=)</code>	1 / 2 / 3	Number of columns in the legend
<code>ax.grid(axis=)</code>	'both' / 'x' / 'y'	Which axes get grid lines
<code>ax.grid(alpha=)</code>	0.2 / 0.5 / 1.0	Grid line transparency
<code>fig.set_facecolor()</code>	'#F8F8F8' / 'white'	Background colour of the whole figure
<code>ax.set_facecolor()</code>	'#FAFAFA'	Background colour of the plot area only
<code>ax.tick_params(axis=, labelsize=)</code>	axis='both', labelsize=10	Controls tick mark label size
<code>ax.spines['top'].set_visible()</code>	False	Hides the top border of the chart
<code>ax.spines['right'].set_visible()</code>	False	Hides the right border of the chart

2

Task

Practice — Multi-Line Chart

Build a multi-line chart that shows only the NORTH, EAST, and SOUTH regions (3 lines). Filter the DataFrame first: `df_top = df_sales[df_sales['region'].isin(['North', 'East', 'South'])]`

Requirements:

- Assign a distinct colour and marker to each of the 3 lines

- Add a legend inside the chart at 'upper left'
- Add a horizontal dashed line at the overall mean of the 3 selected regions
- Hide the top and right chart borders using `ax.spines`
- Format the y-axis with £ notation



Section 4 — Bar Chart

Compare totals or aggregates across categories

4a. Anatomy of a Bar Chart

Chart Part	What it is	Set with
Bar	A vertical rectangle representing a single value	<code>ax.bar(x, height)</code>
Bar width	Fraction of available space each bar occupies	<code>width=0.6</code>
Bar colour	Fill colour per bar (list = different per bar)	<code>color=['#2196F3', '#4CAF50', ...]</code>
Edge colour	Outline drawn around each bar	<code>edgecolor='white' or 'black'</code>
Value label	Text printed above each bar showing its value	<code>ax.bar_label(bars, fmt='£%.0f')</code>
Y-axis limit	Max height so labels are not cut off at top	<code>ax.set_ylim(0, max*1.15)</code>
X-axis ticks	Text labels under each bar	<code>ax.set_xticks / set_xticklabels</code>

4b. Complete Code Template — Bar Chart

```
In [ ]: Vertical bar chart — total sales by region
# — Bar Chart — Total Sales by Region —————

colours = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']

# Step 1 — Create canvas
fig, ax = plt.subplots(figsize=(10, 6))

# Step 2 — Draw bars
bars = ax.bar(
    df_sales['region'], # x — category labels
    df_sales['total_sales'], # y — bar heights
    color=colours,
    width=0.6,           # bar width (0-1)
```



```

    edgecolor='white', # border around each bar
    linewidth=1.5
)

# Add value labels above each bar
ax.bar_label(bars, fmt='£%.0f',
             padding=5, fontsize=10, fontweight='bold')

# Step 3 - Labels and title
ax.set_title('Total H1 2024 Sales by Region',
            fontsize=15, fontweight='bold', pad=14)
ax.set_xlabel('Region', fontsize=12)
ax.set_ylabel('Total Sales (£)', fontsize=12)

# Step 4 - Formatting
ax.yaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax.set_ylim(0, df_sales['total_sales'].max() * 1.18)
ax.grid(axis='y', alpha=0.3, linestyle='--')
ax.spines[['top', 'right']].set_visible(False)

# Step 5 - Render
plt.tight_layout()
plt.show()

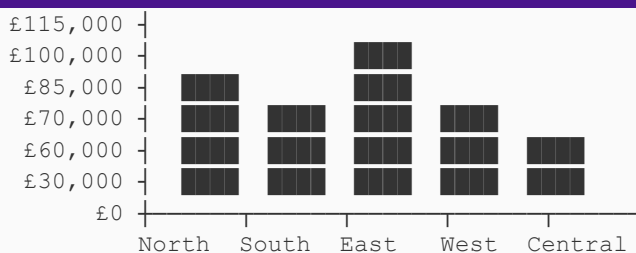
```

Out[:]: output
[Vertical bar chart rendered - 5 coloured bars]

Region	Total Sales (£)	Relative Height
East	£97,600	← tallest
North	£85,000	
Central	£79,400	
South	£66,600	
West	£59,000	← shortest

Value labels bold above each bar. Y-axis £ formatted. Top/right borders hidden.

Chart Preview — Vertical Bar Chart — Total Sales by Region



Value labels (£ formatted) appear bold above each bar. Top and right borders are hidden.

4c. Bar Chart — Property Reference

Property / Argument	Example Value(s)	What it controls
<code>ax.bar(x, height)</code>	<code>bar(regions, totals)</code>	Core call: x = categories, height = values

<code>width=</code>	0.4 / 0.6 / 0.8	Bar width as fraction of category space
<code>color=</code>	'steelblue' / ['#2196F3',...]	Single colour or list (one per bar)
<code>edgecolor=</code>	'white' / 'black' / 'none'	Border colour drawn around each bar
<code>linewidth=</code>	0.5 / 1.5 / 2	Border thickness
<code>alpha=</code>	0.7 / 1.0	Bar transparency
<code>ax.bar_label(bars,fmt=)</code>	fmt='£%.0f' / fmt='%.1f%%'	Auto-labels bars; fmt controls number format
<code>ax.bar_label(padding=)</code>	padding=4 / 8	Pixels of space between bar top and label
<code>ax.bar_label(fontsize=)</code>	fontsize=9 / 11	Font size of value label text
<code>ax.barh(y,width)</code>	barh(regions, totals)	Horizontal bar chart (x and y axes swapped)
<code>ax.set_ylim(0, max*1.2)</code>	set_ylim(0, 115000)	Leaves room for value labels at top
<code>ax.spines[list].set_visible(False)</code>	['top','right']	Hides chart border lines for clean look

3

Task

Practice — Bar Chart

Build a HORIZONTAL bar chart (`ax.barh`) showing `total_sales` per region, sorted ascending.

Tip: sort first with `df_sorted = df_sales.sort_values('total_sales')` before plotting.

Requirements:

- Use `ax.barh()` instead of `ax.bar()`
- Sort bars from lowest (top) to highest (bottom) — ascending sort, then plot
- Use `ax.bar_label()` to add value labels to the right of each bar
- Format x-axis with £ notation (use `ax.xaxis.set_major_formatter`)
- Title: 'Regional H1 2024 Sales — Ranked Lowest to Highest'
- Hide top and right spines



Section 5 — Pie Chart

Show proportional share of a whole

5a. Anatomy of a Pie Chart

Chart Part	What it is	Set with
------------	------------	----------

Wedge	A slice of the pie representing one category	<code>ax.pie()</code> draws all wedges at once
Label	Category name outside the pie	<code>labels=df['region']</code>
Auto-pct	Percentage shown inside or on each wedge	<code>autopct='%1.1f%%'</code>
Explode	Offsets a wedge outward to emphasise it	<code>explode=[0.1, 0, 0, 0, 0]</code>
Shadow	Drop shadow below the chart	<code>shadow=True</code>
Start angle	Angle where the first wedge begins	<code>startangle=90</code> (starts at top)
Pct distance	Radial position of the % label	<code>pctdistance=0.82</code> (inside near edge)

💡 Pro Tip

Use pie charts ONLY when you have 2–6 slices and they represent parts of a whole (they must sum to 100%).

If you have more than 6 categories, a bar chart is almost always easier to read.

Always call `plt.axis('equal')` to ensure the pie is circular, not oval.

Use `pctdistance=` to control whether % labels sit inside or outside the pie.

5b. Complete Code Template — Pie Chart

In []: *Pie chart — revenue share by region*

```
# — Pie Chart — Revenue Share by Region —————

colours = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']
explode = [0.07, 0.04, 0.04, 0.04, 0.04] # emphasise East

# Step 1 — Create canvas (square = circular pie)
fig, ax = plt.subplots(figsize=(8, 8))

# Step 2 — Draw the pie
wedges, texts, autotexts = ax.pie(
    df_sales['total_sales'], # slice sizes
    labels = df_sales['region'], # outer labels
    autopct = '%1.1f%%', # % inside slices
    pctdistance = 0.82, # % label position (0=centre, 1=edge)
    colors = colours,
    explode = explode,
    startangle = 90, # first wedge at top (12 o'clock)
    shadow = True,
    wedgeprops = {'linewidth': 2, 'edgecolor': 'white'}
)

# Style the percentage text inside wedges
```

```

for at in autotexts:
    at.set_fontsize(11)
    at.set_fontweight('bold')
    at.set_color('white')

# Step 3 – Title
ax.set_title('H1 2024 Revenue Share by Region',
             fontsize=15, fontweight='bold', pad=20)

# Step 4 – Equal axes = circular (not oval)
plt.axis('equal')

# Step 5 – Render
plt.tight_layout()
plt.show()

```

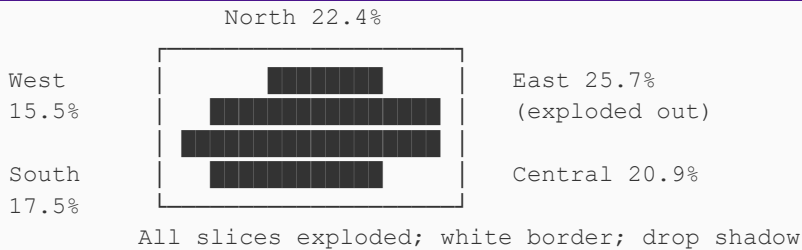
Out[: output

```

[Pie chart rendered – 5 colour-coded slices]
East      25.7%    (£97,600)    ← largest & exploded out
North     22.4%    (£85,000)
Central   20.9%    (£79,400)
South     17.5%    (£66,600)
West      15.5%    (£59,000)    ← smallest
White edges between slices. Drop shadow. % labels bold white inside slices.
Total H1 revenue represented: £387,600

```

Chart Preview — Pie Chart — Revenue Share by Region



i Exploded East slice draws attention to the top performer. % labels in bold white inside each slice.

5c. Pie Chart — Property Reference

Property / Argument	Example Value(s)	What it controls
labels=	df['region'] / ['A','B','C']	Category names shown outside each wedge
autopct=	'%1.0f%%' / '%1.1f%%' / '%1.2f%%'	Format for % displayed on slices (1 decimal = %1.1f%%)
pctdistance=	0.5 / 0.82 / 1.1	0=centre, 1=edge of pie, >1=outside pie
labeldistance=	0.9 / 1.1 / 1.3	Distance of outer text label from centre

<code>colors=</code>	<code>['#2196F3', '#4CAF50', ...]</code>	List of colours, one per wedge
<code>explode=</code>	<code>[0.1, 0, 0, 0, 0]</code>	Offset each slice (0=flush, 0.1=pull out 10%)
<code>startangle=</code>	<code>0 / 90 / 180</code>	Angle where first wedge starts (90 = top)
<code>shadow=</code>	<code>True / False</code>	Drop shadow beneath the pie
<code>wedgeprops=</code>	<code>{'linewidth':2, 'edgecolor':'white'}</code>	Dict of properties applied to every wedge
<code>counterclock=</code>	<code>True / False</code>	Direction slices are drawn (default anti-clockwise)
<code>plt.axis('equal')</code>	<code>—</code>	Required: makes the pie circular
<code>at.set_color()</code>	<code>'white' / 'black'</code>	Colour of the autopct % text labels

4

Task

Practice — Pie Chart

Build a pie chart showing Q1 sales only (jan + feb + mar) per region.

```
df_sales['q1_sales'] = df_sales[['jan', 'feb', 'mar']].sum(axis=1)
```

Requirements:

- Use a different colour palette from the example (try pastels or muted tones)
- Explode the WEST slice (lowest) to draw attention to the laggard
- Set autopct to show 1 decimal place
- Place % labels outside the pie using pctdistance=1.1
- Title: 'Q1 2024 Revenue Share by Region (Jan–Mar)'
- Call `plt.axis('equal')` to ensure circular shape

◆ Section 6 — Scatter Plot

Explore relationships between two numeric variables

6a. Anatomy of a Scatter Plot

Chart Part	What it is	Set with
Point	A dot drawn at the intersection of (x, y)	<code>ax.scatter(x, y)</code>
Point size (s=)	Area of the dot in points ²	<code>s=50 / s=150 / s=[50,100,200]</code>
Point colour	Fill colour of each dot	<code>c='#2196F3'</code> or <code>c=[list of colours]</code>
Marker	Shape of each dot	<code>marker='o' / 'D' / '^' / 's'</code>

Alpha	Transparency — useful when points overlap	<code>alpha=0.6</code>
Annotation	Text label placed near a specific point	<code>ax.annotate('label', (x,y))</code>
Trend line	Dashed line showing the general direction	<code>plot()</code> with a straight line (manual)
Colour map	Map a 3rd numeric variable to point colour	<code>c=values, cmap='viridis'</code>

6b. Complete Code Template — Scatter Plot

In []: *Scatter plot — January sales vs total H1 sales*

```
# — Scatter Plot — Jan sales vs Total H1 —————

colours = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']

# Step 1 — Create canvas
fig, ax = plt.subplots(figsize=(9, 6))

# Step 2 — Draw one point per region
for i, row in df_sales.iterrows():
    ax.scatter(
        row['jan'],          # x — January sales
        row['total_sales'],  # y — H1 total
        color=colours[i],
        s=200,               # dot size (area in points²)
        zorder=5,           # draw on top of grid lines
        edgecolors='white',
        linewidths=2
    )
    # Annotate each point with region name
    ax.annotate(
        row['region'],
        xy=(row['jan'], row['total_sales']),
        textcoords='offset points',
        xytext=(9, 3),      # pixels right and up
        fontsize=10,
        fontweight='bold'
    )

# Step 3 — Labels and title
ax.set_title('January Sales vs Total H1 Sales',
             fontsize=14, fontweight='bold', pad=12)
ax.set_xlabel('January Sales (£)', fontsize=12)
ax.set_ylabel('Total H1 Sales (£)', fontsize=12)

# Step 4 — Formatting
```

```
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax.grid(alpha=0.3, linestyle='--')
ax.spines[['top', 'right']].set_visible(False)
```

```
# Step 5 – Render
plt.tight_layout()
plt.show()
```

Out[: output

```
[Scatter plot rendered – 5 annotated coloured points]
East      (15200, 97600) – top-right ← strongest correlation
North     (12400, 85000)
Central   (11300, 79400)
South     ( 9800, 66600)
West      ( 8700, 59000) – bottom-left ← lowest performer
Strong positive linear pattern: higher Jan → higher H1 total.
Both axes formatted with £ and comma separators.
```

Chart Preview — Scatter Plot — January vs Total H1 Sales



Each dot is annotated with region name. Strong positive correlation visible — Jan performance predicts H1 total.

6c. Scatter Plot — Property Reference

Property / Argument	Example Value(s)	What it controls
<code>ax.scatter(x,y)</code>	<code>scatter(df['jan'], df['total_sales'])</code>	Plots points at (x,y) coordinates
<code>s=</code>	50 / 150 / 200 / [list]	Point size in points ² (list = variable size)
<code>c= / color=</code>	'#2196F3' / [list of colours]	Dot fill colour; list = one colour per point
<code>marker=</code>	'o' / 'D' / '^' / 's' / '+'	Shape of the data point marker
<code>alpha=</code>	0.4 / 0.7 / 1.0	Point transparency
<code>edgecolors=</code>	'white' / 'black' / 'none'	Border colour drawn around each dot
<code>linewidths=</code>	1 / 2 / 3	Thickness of the dot border

<code>zorder=</code>	1 / 3 / 5	Draw order — higher = drawn on top of lower
<code>ax.annotate(text,xy=,xytext=)</code>	<code>annotate('East', xy=(x,y), xytext=(9,3))</code>	Adds a text label near a point
<code>cmap=</code>	'viridis' / 'plasma' / 'RdYlGn'	Colour map — maps a 3rd numeric to colour
<code>c= (colourmap mode)</code>	<code>c=df['total_sales'], cmap='viridis'</code>	Colours points by a 3rd numeric variable
<code>ax.colorbar()</code>	<code>plt.colorbar(scatter)</code>	Adds a colour scale legend for cmap charts

5

Task

Practice — Scatter Plot

Build a scatter plot comparing MARCH sales (x) vs JUNE sales (y) for all 5 regions.

Requirements:

- Plot each region as a separate point (loop with `iterrows()`)
- Use a different colour per region
- Annotate each point with the region name using `ax.annotate()`
- Format both axes with £ notation using `mticker.FuncFormatter`
- Add a reference line using `ax.axline()` or `ax.plot()` showing the 1:1 line ($x=y$)
- Title: 'March vs June Sales — Regional Comparison'

Note: the 1:1 line shows whether June was better than March for each region



Section 7 — Chart Formatting Master Reference

Every formatting option in one place

7a. Title, Labels and Fonts

Property / Argument	Example Value(s)	What it controls
<code>ax.set_title(text)</code>	<code>set_title('H1 2024 Sales')</code>	Main chart heading
<code>fontsize=</code>	10 / 13 / 16	Text size in points
<code>fontweight=</code>	'bold' / 'normal' / 'light'	Text weight
<code>color=</code>	'navy' / '#1A237E'	Text colour
<code>pad=</code>	8 / 14 / 20	Space between title and chart in points
<code>loc=</code>	'left' / 'center' / 'right'	Horizontal alignment of title

<code>ax.set_xlabel(text, fontsize=)</code>	<code>set_xlabel('Month', fontsize=12)</code>	X-axis label
<code>ax.set_ylabel(text, fontsize=)</code>	<code>set_ylabel('Sales (£)', fontsize=12)</code>	Y-axis label
<code>ax.tick_params(labelsize=)</code>	<code>tick_params(labelsize=10)</code>	Tick number size on both axes
<code>ax.set_xticklabels(labels, rotation=)</code>	<code>set_xticklabels(months, rotation=45)</code>	Rotate x-axis labels
<code>plt.suptitle(text, y=)</code>	<code>suptitle('Dashboard', y=1.02)</code>	Overall title above all subplots

7b. Axes, Limits, and Ticks

Property / Argument	Example Value(s)	What it controls
<code>ax.set_xlim(min, max)</code>	<code>set_xlim(0, 7)</code>	Fix horizontal axis range
<code>ax.set_ylim(min, max)</code>	<code>set_ylim(0, 20000)</code>	Fix vertical axis range
<code>ax.invert_yaxis()</code>	<code>invert_yaxis()</code>	Flip y-axis (useful for ranked bar charts)
<code>ax.set_xticks(positions)</code>	<code>set_xticks([0,1,2,3,4,5])</code>	Set where tick marks appear
<code>ax.set_xticklabels(labels)</code>	<code>set_xticklabels(['Jan', 'Feb', ...])</code>	Set text at each tick mark
<code>ax.yaxis.set_major_formatter()</code>	<code>FuncFormatter(lambda x, _: f'£{x:,.0f}')</code>	Custom format for y-axis tick labels
<code>ax.xaxis.set_major_formatter()</code>	<code>FuncFormatter(lambda x, _: f'{x:.1f}%')</code>	Custom format for x-axis tick labels
<code>ax.tick_params(axis=, length=)</code>	<code>tick_params(axis='x', length=5)</code>	Control tick mark length
<code>ax.tick_params(axis=, direction=)</code>	<code>direction='in' / 'out' / 'inout'</code>	Tick direction relative to chart

7c. Grid, Spines, and Background

Property / Argument	Example Value(s)	What it controls
<code>ax.grid(True)</code>	<code>grid(True)</code> / <code>grid(False)</code>	Toggle grid lines on/off
<code>ax.grid(axis=)</code>	<code>axis='y'</code> / <code>'x'</code> / <code>'both'</code>	Which axes show grid lines
<code>ax.grid(alpha=)</code>	<code>alpha=0.2</code> / <code>0.5</code>	Grid line transparency
<code>ax.grid(linestyle=)</code>	<code>ls='--'</code> / <code>':'</code> / <code>'-'</code>	Grid line style
<code>ax.grid(linewidth=)</code>	<code>linewidth=0.5</code> / <code>1</code>	Grid line thickness
<code>ax.grid(color=)</code>	<code>color='gray'</code> / <code>'#CCCCCC'</code>	Grid line colour
<code>ax.spines['top'].set_visible(False)</code>	<code>set_visible(False)</code>	Hide the top chart border
<code>ax.spines['right'].set_visible(False)</code>	<code>set_visible(False)</code>	Hide the right chart border
<code>ax.spines['left'].set_linewidth(2)</code>	<code>set_linewidth(2)</code>	Thicken the left border
<code>ax.set_facecolor()</code>	<code>set_facecolor('#F5F5F5')</code>	Background colour of the plot area
<code>fig.set_facecolor()</code>	<code>set_facecolor('white')</code>	Background colour of the full figure
<code>plt.style.use()</code>	<code>plt.style.use('seaborn-v0_8-whitegrid')</code>	Apply a built-in theme to all charts

7d. Legend and Annotations

Property / Argument	Example Value(s)	What it controls
<code>ax.legend()</code>	<code>legend()</code>	Auto-collects all label= entries into a legend box
<code>ax.legend(loc=)</code>	<code>'best'</code> / <code>'upper left'</code> / <code>'lower right'</code>	Position of the legend box
<code>ax.legend(fontsize=)</code>	<code>fontsize=9</code> / <code>11</code>	Size of legend text
<code>ax.legend(title=)</code>	<code>title='Region'</code>	Heading inside the legend box
<code>ax.legend(frameon=)</code>	<code>frameon=False</code>	Remove the box border around legend
<code>ax.legend(framealpha=)</code>	<code>0.5</code> / <code>0.9</code>	Legend box background transparency

<code>ax.legend(ncol=)</code>	<code>ncol=2 / 3</code>	Columns in legend (useful for many items)
<code>ax.axhline(y=,ls=,color=)</code>	<code>axhline(14000, ls='--', color='red')</code>	Horizontal reference line
<code>ax.axvline(x=,ls=,color=)</code>	<code>axvline(3, ls=':', color='blue')</code>	Vertical reference line
<code>ax.annotate(text,xy=,xytext=)</code>	<code>annotate('Peak', xy=(5,18500))</code>	Text label near a data point
<code>ax.text(x,y,text)</code>	<code>ax.text(2, 15000, 'Q1 End')</code>	Free-position text anywhere on chart
<code>ax.fill_between(x,y1,y2)</code>	<code>fill_between(months, low, high, alpha=0.2)</code>	Shaded band between two y values

7e. Figure Layout and Saving

Property / Argument	Example Value(s)	What it controls
<code>plt.subplots(figsize=(w,h))</code>	<code>figsize=(10,6)</code>	Width × height in inches (72 DPI default)
<code>plt.subplots(1,2)</code>	<code>subplots(1,2,figsize=(14,5))</code>	1 row, 2 side-by-side charts
<code>plt.subplots(2,2)</code>	<code>subplots(2,2,figsize=(12,10))</code>	2×2 grid of 4 charts
<code>plt.tight_layout()</code>	<code>tight_layout()</code>	Auto-adjusts margins to prevent overlap
<code>plt.tight_layout(pad=)</code>	<code>tight_layout(pad=2.0)</code>	Extra padding around all subplots
<code>plt.subplots_adjust(hspace=)</code>	<code>subplots_adjust(hspace=0.4)</code>	Vertical gap between subplot rows
<code>plt.subplots_adjust(wspace=)</code>	<code>subplots_adjust(wspace=0.3)</code>	Horizontal gap between subplot columns
<code>plt.savefig(filename,dpi=)</code>	<code>savefig('chart.png', dpi=150)</code>	Save chart as PNG, JPG, SVG, or PDF
<code>plt.savefig(bbox_inches=)</code>	<code>bbox_inches='tight'</code>	Prevents titles/labels being cropped
<code>plt.show()</code>	<code>show()</code>	Render and display the chart in the notebook
<code>plt.close()</code>	<code>close()</code>	Close the figure (frees memory)
<code>plt.style.use('default')</code>	<code>style.use('default')</code>	Reset to default style after changing



Section 8 — Subplots Dashboard

Combine multiple charts in one figure

8a. Subplots — How they work



Key Concept

`plt.subplots(rows, cols)` creates a grid of axes. Each ax in the grid is independent.

`plt.subplots(1,2)` = 1 row, 2 columns → returns fig and (ax1, ax2).

`plt.subplots(2,2)` = 2 rows, 2 columns → returns fig and 2D array of axes.

Call `plt.suptitle()` for an overall figure title above all subplots.

`plt.tight_layout()` handles spacing — call it before `plt.show()` always.

8b. Complete Code Template — 2×2 Dashboard

In []: 2×2 dashboard — bar, line, pie, scatter

```
# — 2×2 Dashboard — 4 charts in one figure —————

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
colours = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']
markers = ['o', 's', '^', 'D', '*']

# Create 2×2 grid — 4 separate axes objects
fig, axes = plt.subplots(2, 2, figsize=(16, 11))
ax1, ax2, ax3, ax4 = axes.flatten()

# — TOP LEFT: Bar Chart —————
bars = ax1.bar(df_sales['region'], df_sales['total_sales'], color=colours,
width=0.6)
ax1.bar_label(bars, fmt='£%.0f', fontsize=8, fontweight='bold')
ax1.set_title('Total Sales by Region', fontweight='bold')
ax1.set_ylabel('Sales (£)')
ax1.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax1.set_ylim(0, df_sales['total_sales'].max() * 1.18)
ax1.grid(axis='y', alpha=0.3, ls='--')

# — TOP RIGHT: Multi-Line Chart —————
for i, row in df_sales.iterrows():
    ax2.plot(months, row[months_cols],
            color=colours[i], marker=markers[i],
            linewidth=2, markersize=6, label=row['region'])
ax2.set_title('Monthly Trend by Region', fontweight='bold')
ax2.set_ylabel('Sales (£)')
ax2.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax2.legend(fontsize=8, loc='upper left')
ax2.grid(alpha=0.25, ls='--')
```

```
# — BOTTOM LEFT: Pie Chart —————
ax3.pie(df_sales['total_sales'], labels=df_sales['region'],
        autopct='%1.1f%%', colors=colours, pctdistance=0.82,
        startangle=90, wedgeprops={'linewidth':2, 'edgecolor':'white'})
ax3.set_title('Revenue Share', fontweight='bold')
ax3.axis('equal')

# — BOTTOM RIGHT: Scatter Plot —————
for i, row in df_sales.iterrows():
    ax4.scatter(row['jan'], row['total_sales'],
                color=colours[i], s=180, zorder=5)
    ax4.annotate(row['region'], xy=(row['jan'],row['total_sales']),
                 textcoords='offset points', xytext=(8,3), fontsize=9)
ax4.set_title('Jan vs Total H1', fontweight='bold')
ax4.set_xlabel('January Sales (£)')
ax4.set_ylabel('Total H1 Sales (£)')
ax4.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'£{x:,.0f}'))
ax4.grid(alpha=0.25, ls='--')

# Overall figure title and layout
plt.suptitle('DataViz Analytics — H1 2024 Sales Dashboard',
             fontsize=16, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```

Out[:]: output

[2x2 dashboard rendered — 4 charts in one figure]

Top-left : Bar chart — 5 coloured bars with value labels

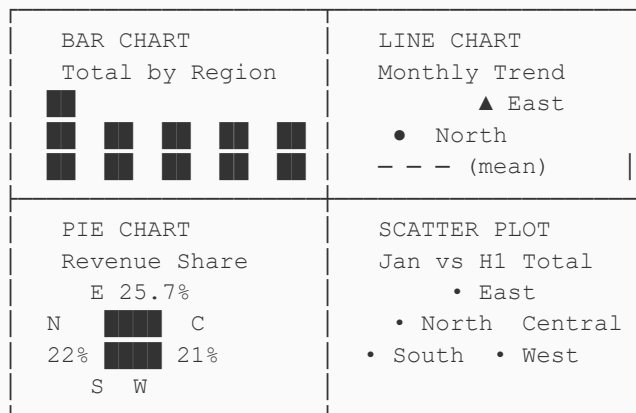
Top-right : Line chart — 5 lines with legend, dashed grid

Bottom-left : Pie chart — 5 slices with % labels, white edges

Bottom-right: Scatter — 5 annotated points, positive correlation

Overall title: DataViz Analytics — H1 2024 Sales Dashboard

Chart Preview — 2x2 Dashboard — All Four Chart Types



i All four chart types in a single 2x2 figure. `plt.suptitle()` adds the overall dashboard heading.

Task	In DA_Guide_7_5.ipynb build a COMPLETE dashboard with all four chart types. Requirements (all in one Code cell using plt.subplots(2,2)): - Top-left: Bar chart — Q1 total sales per region (jan+feb+mar) - Top-right: Line chart — EAST region only, monthly trend with mean line - Bottom-left: Pie chart — June sales share (df_sales['jun']) - Bottom-right: Scatter — May sales (x) vs June sales (y) per region All charts must have: title, labelled axes, £ formatted y-axis Overall subtitle: 'DataViz Analytics Ltd. — H1 2024 Visual Report' Save the figure: plt.savefig('dashboard.png', dpi=150, bbox_inches='tight')
------	---

9. Complete Matplotlib Quick Reference

Every Matplotlib function and property used across all sections, in one master reference table.

Function / Property	Syntax	What it does
plt.subplots()	fig,ax=plt.subplots(figsize=(w,h))	Create single chart canvas
plt.subplots(r,c)	fig, (ax1,ax2)=plt.subplots(1,2)	Create grid of rxc charts
%matplotlib inline	%matplotlib inline	Render charts inside notebook cells
ax.plot()	ax.plot(x,y,color=,marker=,lw=)	Line chart
ax.bar()	ax.bar(x,height,color=,width=)	Vertical bar chart
ax.barh()	ax.barh(y,width,color=)	Horizontal bar chart
ax.bar_label()	ax.bar_label(bars,fmt='£%.0f',padding=4)	Value labels on bars
ax.scatter()	ax.scatter(x,y,c=,s=,alpha=)	Scatter plot
ax.pie()	ax.pie(vals,labels=,autopct=,colors=)	Pie chart
ax.axhline()	ax.axhline(y=val,ls='--',color=)	Horizontal reference line

<code>ax.axvline()</code>	<code>ax.axvline(x=val, ls=':', color=)</code>	Vertical reference line
<code>ax.annotate()</code>	<code>ax.annotate('text', xy=(x, y))</code>	Text label near a point
<code>ax.text()</code>	<code>ax.text(x, y, 'text', fontsize=)</code>	Free-position text on chart
<code>ax.fill_between()</code>	<code>ax.fill_between(x, y1, y2, alpha=0.2)</code>	Shaded area between two lines
<code>ax.set_title()</code>	<code>ax.set_title('text', fontsize=, fontweight=)</code>	Chart title
<code>ax.set_xlabel()</code>	<code>ax.set_xlabel('Month', fontsize=12)</code>	X-axis label
<code>ax.set_ylabel()</code>	<code>ax.set_ylabel('Sales (£)', fontsize=12)</code>	Y-axis label
<code>ax.set_xlim()</code>	<code>ax.set_xlim(0, 7)</code>	Fix x-axis range
<code>ax.set_ylim()</code>	<code>ax.set_ylim(0, 20000)</code>	Fix y-axis range
<code>ax.set_xticks()</code>	<code>ax.set_xticks([0, 1, 2])</code>	Set tick positions
<code>ax.set_xticklabels()</code>	<code>ax.set_xticklabels(['Jan', 'Feb'])</code>	Set tick labels
<code>ax.yaxis.set_major_formatter()</code>	<code>FuncFormatter(lambda x, _: f'£{x:,.0f}')</code>	Format axis tick labels
<code>ax.legend()</code>	<code>ax.legend(loc='best', fontsize=10)</code>	Add legend box
<code>ax.grid()</code>	<code>ax.grid(axis='y', alpha=0.3, ls='--')</code>	Add grid lines
<code>ax.tick_params()</code>	<code>ax.tick_params(labelsize=10)</code>	Control tick appearance
<code>ax.spines[].set_visible()</code>	<code>ax.spines['top'].set_visible(False)</code>	Hide chart border
<code>ax.set_facecolor()</code>	<code>ax.set_facecolor('#F5F5F5')</code>	Chart area background colour
<code>plt.suptitle()</code>	<code>plt.suptitle('Title', y=1.02)</code>	Overall title for subplots figure

<code>plt.tight_layout()</code>	<code>plt.tight_layout(pad=2.0)</code>	Auto-fix margins and spacing
<code>plt.axis('equal')</code>	<code>plt.axis('equal')</code>	Ensure pie chart is circular
<code>plt.savefig()</code>	<code>plt.savefig('chart.png',dpi=150)</code>	Save chart to file
<code>plt.show()</code>	<code>plt.show()</code>	Render and display the chart
<code>plt.style.use()</code>	<code>plt.style.use('seaborn-v0_8-whitegrid')</code>	Apply built-in theme